Uso de Enums en Java

Vamos a implementar el uso de enums en este código para que el atributo marca de los vehículos se cargue a partir de un Enum de java que definiremos aparte. Actualicemos el modelo de datos con esta nueva clase.

```
package com.universitas;

/**
 * Clase abstracta que representa un vehículo genérico.
 */
public abstract class Vehiculo {

    private String marca;
    private String modelo;

    /**
     * Constructor base de vehículo.
     *
     * @param marca  marca del vehículo
     * @param modelo modelo del vehículo
     */
    public Vehiculo(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    /**
     * Devuelve la marca del vehículo.
     *
     * @return marca
     */
    public String getMarca() {
        return marca;
    }

    /**
     * Devuelve el modelo del vehículo.
     *
     * @return modelo
     */
```

```
    public String getModelo() {
        return modelo;
    }
//obliga a las clases derivadas a implementar este
método para agregar ruedas, ya que cada tipo de
vehículo puede tener una cantidad diferente de
ruedas.
public abstract void agregarRueda(Rueda rueda);

    /**
     * Muestra la información del vehículo.
     */
    public abstract void mostrarInfo();
}
```

Perfecto, este es un muy buen caso para introducir **Enums en Java** dentro de tu modelo orientado a objetos 👌 (además suma mucho para buenas prácticas y validación de datos).

La idea es:

- Reemplazar `String marca` → `Marca marca`
- Definir un `enum Marca`
- Ajustar constructor, getters y (opcional) `toString()`
- Mantener consistencia en todo el modelo

✅ 1. Nuevo Enum: `Marca`

Creamos una clase aparte dentro del mismo paquete:

Java

```java
package com.universitas;

/**
 * Enum que representa las marcas de vehículos disponibles.
 */
public enum Marca {
    TOYOTA,
    FORD,
    CHEVROLET,
    VOLKSWAGEN,
    RENAULT,
```

```
    PEUGEOT,
    FIAT
}
```

👉 Ventajas:

- Evita errores de tipeo (`"Toyta"`, `"ford"`, etc.)
- Limita valores válidos
- Mejora la legibilidad del modelo

✅ 2. Clase `Vehiculo` actualizada

Ahora reemplazamos `String` por `Marca`:

```java
package com.universitas;

/**
 * Clase abstracta que representa un vehículo genérico.
 */
public abstract class Vehiculo {

    private Marca marca;
    private String modelo;

    /**
     * Constructor base de vehículo.
     *
     * @param marca  marca del vehículo (enum)
     * @param modelo modelo del vehículo
     */
    public Vehiculo(Marca marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    /**
     * Devuelve la marca del vehículo.
     *
     * @return marca
     */
    public Marca getMarca() {
        return marca;
    }

    /**
```

```
     * Devuelve el modelo del vehículo.
     *
     * @return modelo
     */
    public String getModelo() {
        return modelo;
    }

    // obliga a las clases derivadas a implementar este método
    public abstract void agregarRueda(Rueda rueda);

    /**
     * Muestra la información del vehículo.
     */
    public abstract void mostrarInfo();
}
```

✅ 3. Ejemplo de uso (IMPORTANTE para alumnos)

```java
Auto auto = new Auto(Marca.TOYOTA, "Corolla");
```

👉 Ya no podés hacer esto:

```java
new Auto("Toyota", "Corolla"); // ❌ error
```

✅ 4. Mejora opcional (muy recomendable): mostrar nombre amigable

Si querés que no salga en mayúsculas:

```java
public enum Marca {
    TOYOTA("Toyota"),
    FORD("Ford"),
    CHEVROLET("Chevrolet");

    private String nombre;

    Marca(String nombre) {
        this.nombre = nombre;
```

```
    }

    public String getNombre() {
        return nombre;
    }
}
```

Y en `mostrarInfo()`:

```java
System.out.println("Marca: " + marca.getNombre());
```

✅ 5. Impacto en el modelo (explicación didáctica)

Esto introduce un concepto clave:

👉 **Tipo fuerte (Strong Typing)**

Antes:

```java
String marca;
```

Ahora:

```java
Marca marca;
```

✔️ Más seguro
✔️ Más mantenible
✔️ Más orientado a dominio

🔥 6. (Muy bueno para clase) Concepto que estás enseñando

Con este cambio estás introduciendo:

- Modelado de dominio real
- Enumeraciones como objetos
- Restricción de estados válidos
- Clean Code (evitar "magic strings")

Si querés, en el próximo paso podemos:

👉 actualizar **Auto, Camion y tests con JUnit 5**
👉 o incluso hacer validaciones + persistencia JDBC usando enums (muy interesante para tu etapa 2)

¿Seguimos con eso?

agregame el ENum al modelo de clases para Mermaid

Perfecto, vamos a integrar el **enum** `Marca` **dentro del diagrama UML en Mermaid**, respetando tu modelo actual (Vehiculo + Auto + Rueda, etc.) 👇

✅ Diagrama UML actualizado (con Enum)

Mermaid

```
classDiagram

%% ========================
%% ENUM
%% ========================
class Marca {
  <<enumeration>>
  TOYOTA
  FORD
  CHEVROLET
  VOLKSWAGEN
  RENAULT
  PEUGEOT
  FIAT
}

%% ========================
```

🧠 Explicación clave para clase

👉 Lo importante acá:

✔️ Enum como tipo de atributo

```
Vehiculo --> Marca : usa
```

- `Vehiculo` **no hereda**, sino que **usa el enum como tipo**
- Es una **relación de dependencia (uso)**

✔️ Notación `<<enumeration>>`

Indica que `Marca`:

- no es una clase común
- es un conjunto cerrado de valores

✔️ Cambio conceptual fuerte

Antes:

```
marca : String
```

Ahora:

```
marca : Marca
```

💡 Esto es clave para explicar:

- **modelo más fuerte**
- **menos errores**
- **mejor diseño OO**

🚀 Si querés subir un nivel más (muy bueno para alumnos)

Podemos en el siguiente paso:

👉 agregar comportamiento al enum (ej: país de origen)
👉 integrar enums en JDBC (cómo persistirlos)
👉 o hacer un diagrama extendido con `Camion` y `Vehiculo` polimórfico

Solo decime 👍

dame el código completo del Enum con la mejora amigable y el código completo de la clase Vehiculo

Perfecto, te dejo ambas clases completas, listas para usar y bien documentadas para tu proyecto 👇

✅ Enum `Marca` (con nombre amigable)

Java

```java
package com.universitas;

/**
 * Enum que representa las marcas de vehículos disponibles.
 * Incluye un nombre amigable para mostrar por pantalla.
 */
public enum Marca {

    TOYOTA("Toyota"),
    FORD("Ford"),
    CHEVROLET("Chevrolet"),
    VOLKSWAGEN("Volkswagen"),
    RENAULT("Renault"),
    PEUGEOT("Peugeot"),
    FIAT("Fiat");

    private String nombre;

    /**
     * Constructor del enum.
     *
     * @param nombre Nombre amigable de la marca
     */
    Marca(String nombre) {
        this.nombre = nombre;
    }

    /**
     * Devuelve el nombre amigable de la marca.
     *
     * @return nombre de la marca
     */
    public String getNombre() {
        return nombre;
    }
```

```java
    /**
     * Sobrescribe toString para mostrar el nombre amigable.
     *
     * @return nombre de la marca
     */
    @Override
    public String toString() {
        return nombre;
    }
}
```

✅ Clase `Vehiculo` actualizada

```java
package com.universitas;

/**
 * Clase abstracta que representa un vehículo genérico.
 */
public abstract class Vehiculo {

    private Marca marca;
    private String modelo;

    /**
     * Constructor base de vehículo.
     *
     * @param marca  marca del vehículo (enum Marca)
     * @param modelo modelo del vehículo
     */
    public Vehiculo(Marca marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
    }

    /**
     * Devuelve la marca del vehículo.
     *
     * @return marca
     */
    public Marca getMarca() {
        return marca;
    }

    /**
```

```java
     * Devuelve el modelo del vehículo.
     *
     * @return modelo
     */
    public String getModelo() {
        return modelo;
    }

    /**
     * Método abstracto para agregar ruedas.
     * Cada tipo de vehículo implementa su propia lógica.
     *
     * @param rueda rueda a agregar
     */
    public abstract void agregarRueda(Rueda rueda);

    /**
     * Muestra la información del vehículo.
     * Debe ser implementado por las clases hijas.
     */
    public abstract void mostrarInfo();
}
```

🔥 Tip didáctico (para remarcar en clase)

Gracias al `toString()` del enum:

```java
System.out.println(auto.getMarca());
```

👉 ya imprime:

```
Toyota
```

(en lugar de `TOYOTA`)

Si querés, en el próximo paso te actualizo:

- ✅ `Auto` con este cambio
- ✅ los **tests JUnit 5** (esto cambia bastante porque ahora comparás enums)
- ✅ o cómo persistir `Marca` en JDBC (muy bueno para tu etapa 2)

Decime cómo seguimos 👍

actualizamos las clases AutoTest y CamionTest (adjunto clases originales): package com.universitas;

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class AutoTest {

    @Test
    void testConstructorAutoValido() {
        Auto auto = new Auto("Toyota", "Corolla", "Nafta",
1800);

        assertAll(
            () -> assertNotNull(auto),
            () -> assertEquals("Toyota", auto.getMarca()),
            () -> assertEquals("Corolla", auto.getModelo()),
            () -> assertNotNull(auto.getMotor()),
            () -> assertEquals("Nafta",
auto.getMotor().getTipo()),
            () -> assertEquals(1800,
auto.getMotor().getCilindrada()),
            () -> assertNotNull(auto.getRuedas()),
            () -> assertEquals(0, auto.getRuedas().size())
        );
    }

    @Test
    void
testAgregarHastaCincoRuedasDeMismaMedida() {
        Auto auto = new Auto("Ford", "Fiesta", "Nafta",
1600);

        for (int i = 0; i < 5; i++) {
            auto.agregarRueda(new Rueda("Pirelli", 15));
        }

        assertEquals(5, auto.getRuedas().size());
    }
```

```
    @Test
    void testAgregarMasDeCincoRuedasDebeFallar() {
        Auto auto = new Auto("Ford", "Focus", "Nafta",
2000);

        for (int i = 0; i < 5; i++) {
            auto.agregarRueda(new Rueda("Pirelli", 15));
        }

        IllegalStateException ex =
assertThrows(IllegalStateException.class,
            () -> auto.agregarRueda(new Rueda("Pirelli",
15)));

        assertTrue(ex.getMessage().contains("más de 5
ruedas"));
    }

    @Test
    void testAgregarRuedaNulaDebeFallar() {
        Auto auto = new Auto("Ford", "Ka", "Nafta", 1000);

        IllegalArgumentException ex =
assertThrows(IllegalArgumentException.class,
            () -> auto.agregarRueda(null));

        assertTrue(ex.getMessage().contains("no puede
ser nula"));
    }

    @Test
    void
testAgregarRuedasDeDistintaMedidaDebeFallar() {
        Auto auto = new Auto("Ford", "Fiesta", "Nafta",
1600);
        auto.agregarRueda(new Rueda("Pirelli", 15));

        IllegalArgumentException ex =
assertThrows(IllegalArgumentException.class,
            () -> auto.agregarRueda(new Rueda("Pirelli",
17)));
```

```
        assertTrue(ex.getMessage().contains("misma
medida"));
    }
} package com.universitas;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CamionTest {

    @Test
    void testConstructorCamionValido() {
        Camion camion = new Camion("Scania", "R450",
"Diesel", 13000, 20000);

        assertNotNull(camion);
        assertEquals("Scania", camion.getMarca());
        assertEquals("R450", camion.getModelo());

        assertEquals(20000, camion.getCapacidadCarga());

        assertNotNull(camion.getMotor());
        assertEquals("Diesel",
camion.getMotor().getTipo());
        assertEquals(13000,
camion.getMotor().getCilindrada());

        assertNotNull(camion.getRuedas());
        assertEquals(0, camion.getRuedas().size());
    }

    @Test
    void testAgregarRuedasCamion() {
        Camion camion = new Camion("Mercedes",
"Actros", "Diesel", 12000, 18000);

        for (int i = 0; i < 6; i++) {
            camion.agregarRueda(new Rueda("Michelin",
22));
        }
```

```
        assertEquals(6, camion.getRuedas().size());
    }

    @Test
    void testAgregarMasDeDiezRuedasDebeFallar() {
        Camion camion = new Camion("Iveco", "Hi-Way",
"Diesel", 13000, 22000);

        for (int i = 0; i < 10; i++) {
            camion.agregarRueda(new Rueda("Pirelli", 22));
        }

        Exception ex =
assertThrows(IllegalStateException.class, () -> {
            camion.agregarRueda(new Rueda("Pirelli", 22));
        });

        assertTrue(ex.getMessage().contains("más de 10
ruedas"));
    }

    @Test
    void testAgregarRuedaNulaDebeFallar() {
        Camion camion = new Camion("Volvo", "FH",
"Diesel", 14000, 25000);

        assertThrows(IllegalArgumentException.class, () ->
{
            camion.agregarRueda(null);
        });
    }
}
```

Sí. Como ahora `marca` es un `enum Marca`, los tests deben crear objetos usando `Marca.TOYOTA`, `Marca.FORD`, etc., y comparar contra el enum, no contra un `String`.

AutoTest actualizado

Java

```java
package com.universitas;

import org.junit.jupiter.api.Test;
```

```java
import static org.junit.jupiter.api.Assertions.*;

class AutoTest {

    @Test
    void testConstructorAutoValido() {
        Auto auto = new Auto(Marca.TOYOTA, "Corolla", "Nafta", 1800);

        assertAll(
            () -> assertNotNull(auto),
            () -> assertEquals(Marca.TOYOTA, auto.getMarca()),
            () -> assertEquals("Toyota", auto.getMarca().getNombre()),
            () -> assertEquals("Corolla", auto.getModelo()),
            () -> assertNotNull(auto.getMotor()),
            () -> assertEquals("Nafta", auto.getMotor().getTipo()),
            () -> assertEquals(1800, auto.getMotor().getCilindrada()),
            () -> assertNotNull(auto.getRuedas()),
            () -> assertEquals(0, auto.getRuedas().size())
        );
    }

    @Test
    void testAgregarHastaCincoRuedasDeMismaMedida() {
        Auto auto = new Auto(Marca.FORD, "Fiesta", "Nafta", 1600);

        for (int i = 0; i < 5; i++) {
            auto.agregarRueda(new Rueda("Pirelli", 15));
        }

        assertEquals(5, auto.getRuedas().size());
    }

    @Test
    void testAgregarMasDeCincoRuedasDebeFallar() {
        Auto auto = new Auto(Marca.FORD, "Focus", "Nafta", 2000);

        for (int i = 0; i < 5; i++) {
            auto.agregarRueda(new Rueda("Pirelli", 15));
        }

        IllegalStateException ex = assertThrows(
            IllegalStateException.class,
            () -> auto.agregarRueda(new Rueda("Pirelli", 15))
        );

        assertTrue(ex.getMessage().contains("más de 5 ruedas"));
    }

    @Test
```

```java
    void testAgregarRuedaNulaDebeFallar() {
        Auto auto = new Auto(Marca.FORD, "Ka", "Nafta", 1000);

        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> auto.agregarRueda(null)
        );

        assertTrue(ex.getMessage().contains("no puede ser nula"));
    }

    @Test
    void testAgregarRuedasDeDistintaMedidaDebeFallar() {
        Auto auto = new Auto(Marca.FORD, "Fiesta", "Nafta", 1600);

        auto.agregarRueda(new Rueda("Pirelli", 15));

        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> auto.agregarRueda(new Rueda("Pirelli", 17))
        );

        assertTrue(ex.getMessage().contains("misma medida"));
    }
}
```

CamionTest actualizado

Java

```java
package com.universitas;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CamionTest {

    @Test
    void testConstructorCamionValido() {
        Camion camion = new Camion(Marca.SCANIA, "R450", "Diesel", 13000, 20000);

        assertAll(
            () -> assertNotNull(camion),
            () -> assertEquals(Marca.SCANIA, camion.getMarca()),
            () -> assertEquals("Scania", camion.getMarca().getNombre()),
            () -> assertEquals("R450", camion.getModelo()),
            () -> assertEquals(20000, camion.getCapacidadCarga()),
            () -> assertNotNull(camion.getMotor()),
```

```
            () -> assertEquals("Diesel", camion.getMotor().getTipo()),
            () -> assertEquals(13000, camion.getMotor().getCilindrada()),
            () -> assertNotNull(camion.getRuedas()),
            () -> assertEquals(0, camion.getRuedas().size())
        );
    }

    @Test
    void testAgregarRuedasCamion() {
        Camion camion = new Camion(Marca.MERCEDES_BENZ, "Actros", "Diesel", 12000, 18000);

        for (int i = 0; i < 6; i++) {
            camion.agregarRueda(new Rueda("Michelin", 22));
        }

        assertEquals(6, camion.getRuedas().size());
    }

    @Test
    void testAgregarMasDeDiezRuedasDebeFallar() {
        Camion camion = new Camion(Marca.IVECO, "Hi-Way", "Diesel", 13000, 22000);

        for (int i = 0; i < 10; i++) {
            camion.agregarRueda(new Rueda("Pirelli", 22));
        }

        IllegalStateException ex = assertThrows(
            IllegalStateException.class,
            () -> camion.agregarRueda(new Rueda("Pirelli", 22))
        );

        assertTrue(ex.getMessage().contains("más de 10 ruedas"));
    }

    @Test
    void testAgregarRuedaNulaDebeFallar() {
        Camion camion = new Camion(Marca.VOLVO, "FH", "Diesel", 14000, 25000);

        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> camion.agregarRueda(null)
        );

        assertTrue(ex.getMessage().contains("no puede ser nula"));
    }
}
```

Tenés que agregar estas marcas al enum:

```java
SCANIA("Scania"),
MERCEDES_BENZ("Mercedes-Benz"),
IVECO("Iveco"),
VOLVO("Volvo");
```

agreguemos una clase Genérica con dos atributos que permita definir nuevos vehiculos de acuerdo a la aplicación (por ejemplo Moto, Barco, Avión) trabajando de modo más eficiente al no escribir tres clases nuevas sino solo una

Sí, podemos hacerlo con una **clase genérica** que herede de `Vehiculo` y permita definir vehículos nuevos sin crear una clase específica para cada uno.

Por ejemplo:

```java
Moto
Barco
Avion
```

podrían representarse con una sola clase:

```java
VehiculoGenerico<T, U>
```

donde `T` y `U` son tipos variables para dos atributos particulares.

Clase `VehiculoGenerico`

```java
package com.universitas;

import java.util.ArrayList;
import java.util.List;

/**
 * Clase genérica que permite representar distintos tipos de vehículos
 * sin necesidad de crear una clase específica para cada uno.
```

```
 *
 * @param <T> tipo del primer atributo específico
 * @param <U> tipo del segundo atributo específico
 */
public class VehiculoGenerico<T, U> extends Vehiculo {

    private String tipoVehiculo;
    private T atributo1;
    private U atributo2;
    private List<Rueda> ruedas;

    /**
     * Constructor de vehículo genérico.
     *
     * @param marca marca del vehículo
     * @param modelo modelo del vehículo
     * @param tipoVehiculo tipo de vehículo, por ejemplo Moto, Barco o Avión
     * @param atributo1 primer atributo específico
     * @param atributo2 segundo atributo específico
     */
    public VehiculoGenerico(Marca marca, String modelo, String tipoVehiculo, T atributo1, U atributo2) {
        super(marca, modelo);
        this.tipoVehiculo = tipoVehiculo;
        this.atributo1 = atributo1;
        this.atributo2 = atributo2;
        this.ruedas = new ArrayList<>();
    }

    public String getTipoVehiculo() {
        return tipoVehiculo;
    }

    public T getAtributo1() {
        return atributo1;
    }

    public U getAtributo2() {
        return atributo2;
    }

    public List<Rueda> getRuedas() {
        return ruedas;
    }

    /**
     * Agrega una rueda al vehículo genérico.
     *
     * @param rueda rueda a agregar
     */
```

```java
    @Override
    public void agregarRueda(Rueda rueda) {
        if (rueda == null) {
            throw new IllegalArgumentException("La rueda no puede ser nula");
        }

        ruedas.add(rueda);
    }

    /**
     * Muestra la información del vehículo genérico.
     */
    @Override
    public void mostrarInfo() {
        System.out.println("Tipo de vehículo: " + tipoVehiculo);
        System.out.println("Marca: " + getMarca());
        System.out.println("Modelo: " + getModelo());
        System.out.println("Atributo 1: " + atributo1);
        System.out.println("Atributo 2: " + atributo2);
        System.out.println("Cantidad de ruedas: " + ruedas.size());
    }
}
```

Ejemplos de uso

Java

```java
VehiculoGenerico<Integer, String> moto =
        new VehiculoGenerico<>(Marca.HONDA, "XR150", "Moto", 150, "Enduro");

VehiculoGenerico<Double, String> barco =
        new VehiculoGenerico<>(Marca.YAMAHA, "WaveRunner", "Barco", 3.5, "Motor fuera de borda");

VehiculoGenerico<Integer, Boolean> avion =
        new VehiculoGenerico<>(Marca.BOEING, "737", "Avión", 180, true);
```

Marcas que deberías agregar al enum `Marca`

Java

```java
HONDA("Honda"),
YAMAHA("Yamaha"),
BOEING("Boeing");
```

La idea didáctica sería:

```java
VehiculoGenerico<T, U>
```

Donde:

```java
T atributo1
U atributo2
```

pueden cambiar según el tipo de vehículo.

Por ejemplo:

```java
Moto   -> cilindrada, tipoUso
Barco  -> eslora, tipoMotor
Avión  -> cantidadPasajeros, presurizado
```

Esto permite mostrar **genéricos en Java** aplicados al modelo de clases sin duplicar código.